Unlocking Polyhedral Optimization for Kokkos Kernels

A Library–Compiler Co-Design

Ugo Battiston[1], Philippe Clauss[1], Marc Pérache[2]

[1] Inria CAMUS, ICube, Université de Strasbourg
[2] CEA, DAM, DIF, Université Paris-Saclay

Context and problem

Kokkos: one source, many architectures

- Heterogeneous supercomputers: CPUs, GPUs, several vendors.
- **Kokkos**: C++ performance-portability library.
  - `View`: multidimensional arrays, layout per target.
  - **Execution policies**: the iteration space.
  - `parallel_for` + lambda: the loop body.
- Templates lower it to OpenMP loops (CPU) or CUDA kernels (GPU).

```C++
Kokkos::View<double**> A("A", n, m), B("B", n, m);
auto policy = Kokkos::MDRangePolicy<
                        Kokkos::OpenMP,
                        Kokkos::Rank<2>
                      >({1, 1}, {n-1, m-1});

Kokkos::parallel_for("stencil", policy,
  KOKKOS_LAMBDA(long i, long j) {
    B(i, j) = A(i, j) + A(i, j-1) + A(i, j+1)
                      + A(i-1, j) + A(i+1, j);
  });
```

Loops are **regular**: regular iteration space, constant bounds, affine accesses. Ideal polyhedral candidates.

The polyhedral model and Polly

Polyhedral model
Mathematical modeling of nested loops:

- Kernel = **SCoP** (Static Control Part).
- Loop nest = set of integer points
- Dependencies = affine constraints.
- Aggressive loop transformations: **tiling, fusion, skewing, wavefront parallelism**.

Polly : the polyhedral optimizer of LLVM

1. Canonicalization
2. SCoP detection
3. SCoP construction
4. Dependence analysis and scheduling
5. Code generation with runtime guards

Works on LLVM IR: the right level for inlined C++ templates, unlike source-to-source tools.

Problem: Polly cannot see Kokkos loops

After inlining the Kokkos dispatch stack, Polly sees:

- a loop body scattered across a region without boundaries,
- **array dimensions lost**: `A(i, j)` is 1D pointer arithmetic,
- bounds and View sizes reloaded inside the loop (no LICM),
- safety checks and loop versioning that break the CFG,
- the default `MDRangePolicy` tiling: extra loop levels,
- no trace of the execution space (Serial / OpenMP / Cuda).

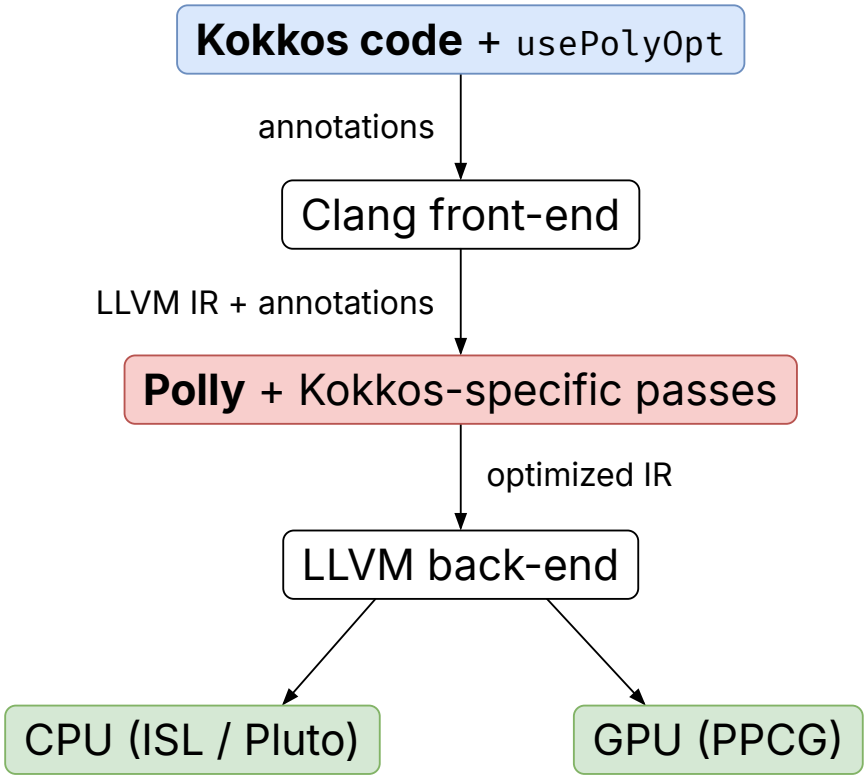
Our claim

The barrier is one of **visibility**, not of optimizer capability.

Our approach

A **library–compiler co-design**: Kokkos emits annotated, clean IR; Polly learns to read it. The user keeps the portable, single-source style.

Contributions at a glance



1. **Kokkos side** : kernels isolations, backend annotations (bounds, View dimensions, backend used), backend simplification, template option to activate the polyhedral path.
2. **Polly side** : annotations extraction, upgraded delinearization, IR cleaning, global view across kernels, triangular loops.
3. **Back-ends** : Pluto ordonnancement, PPCG GPU code generation.

Bridging Kokkos and Polly

A template option for the polyhedral path: usePolyOpt


```C++
Kokkos::parallel_for<Kokkos::usePolyOpt>(policy,
  KOKKOS_LAMBDA(long i, long j) {
    B(i, j) = A(i, j) + A(i, j-1) + A(i, j+1)
                      + A(i-1, j) + A(i+1, j);
  });
```

- Static dispatch between two paths that **coexist** in the library.
- Without the option, Kokkos stays strictly standard.

The polyhedral path

- Leaf functions `noinline` + attribute: Polly scans only them.
- `MDRangePolicy` re-implemented **without its default tiling**: a clean, perfect nest. Polly tiles itself.

Scope

- Policies: `RangePolicy`, `MDRangePolicy`.
- Backends: `Serial`, `OpenMP`, `Cuda`.

Annotations, delinearization and array canonicalization

Annotations :
`__builtin_annotation` → `llvm.annotate` in the IR, emitted by the Kokkos backends.

- policy loop bounds,
- array base pointers bound to their **dimension sizes**,
- the kernel's **execution space**.

They survive reordering or elimination of loads.

Delinearization : `A(i,j,k)` is one SCEV polynomial; Polly guesses the dimension sizes from its terms and **fails on partial accesses** like `A(0,j,k)`. Annotated sizes make it reliable for any access.

Array canonicalization (`ArrayReg2MemPass`) : in SSA form, a loaded value shared between statements becomes a **scalar dependence** that over-constrains the scheduler; Polly's `ForwardOpTree` is too conservative (aliasing).

- Demotes SSA registers of **annotated arrays** back to explicit loads and stores.
- Targets loop-carried, loop-exit, intra-block (reused read, multiple writes) and extended scalar dependences.
- Removes **false dependences**, restores a source-like memory structure.

IR restructuring and backend synchronization

Two cleaning passes in Polly

- **Forced hoisting** of base pointers and dimension reads out of the nest (LICM refuses: possible aliasing).
- **CFG de-noising**: bound checks and loop versioning replaced by `llvm.assume`; Polly constrains the domain and emits one **runtime guard**.

Backend synchronization

- Standard Polly: one target per compilation unit (flag).
- Kokkos: execution space chosen **per kernel**.
- The backend annotation lets Polly choose **per SCoP**:
  - `Serial` kernel → not parallelized,
  - `OpenMP` kernel → OpenMP code generation,
  - `Cuda` kernel → PPCG path.

Complex execution patterns

A global view across kernels: variadic parallel_for

Idiomatic Kokkos (gemver): one `parallel_for` per nest, four separate SCoPs, no fusion.

```cpp
parallel_for<Kokkos::usePolyOpt>(policy2D,
  KOKKOS_LAMBDA(long i, long j) {
    A(i,j) += u1(i)*v1(j) + u2(i)*v2(j);
  });
parallel_for<Kokkos::usePolyOpt>(policy1D,
  KOKKOS_LAMBDA(long i) {
    for (long j = 0; j < n; j++)
      x(i) += beta * A(j,i) * y(j);
  });
parallel_for<Kokkos::usePolyOpt>(policy1D,
  KOKKOS_LAMBDA(long i) {
    x(i) += z(i);
  });
parallel_for<Kokkos::usePolyOpt>(policy1D,
  KOKKOS_LAMBDA(long i) {
    w(i) += alpha * x(i) * y(i);
  });
```

Variadic form: a sequence of (policy, functor) pairs, executed in order.

```cpp
Kokkos::View<"A", double**> A(...); // static name
parallel_for<Kokkos::usePolyOpt>("gemver",
  policy2D, KOKKOS_LAMBDA(long i, long j) {
    A(i,j) += u1(i)*v1(j) + u2(i)*v2(j); },
  policy1D, KOKKOS_LAMBDA(long i) {
    for (long j = 0; j < n; j++)
      x(i) += beta * A(j,i) * y(j);
  },
  policy1D, KOKKOS_LAMBDA(long i) { x(i) += z(i); },
  policy1D, KOKKOS_LAMBDA(long i) {
    w(i) += alpha * A(i,j) * x(j);
});
```

- **CFG linearization**: preheaders hoisted above the sequence, one contiguous region.
- **Static naming**: each lambda captures its own pointer copy; same name → RAUW to the first one.

Relating loop bounds: constraint injection

Each policy keeps its own copy of the bounds; Polly must be told how they relate.

Constraint grammar : `Operand op Operand`

- `pN.lM` / `pN.uM` : bound of dimension `M` of policy `N`,
- `KOKKOS_LOOP_BOUND(n)` : a variable of the body,
- literals;
- Operator : `==`, `<`, `>`, `<=`, `>=`.

```C++
auto policy = Kokkos::RangePolicy(0, n);
parallel_for<Kokkos::usePolyOpt, "p0.u0 == n">(policy,
  KOKKOS_LAMBDA(int i) {
    for (int j = 1; j < KOKKOS_LOOP_BOUND(n) - 1; j++)
      A(i, j) = B(i, j);
  });
```

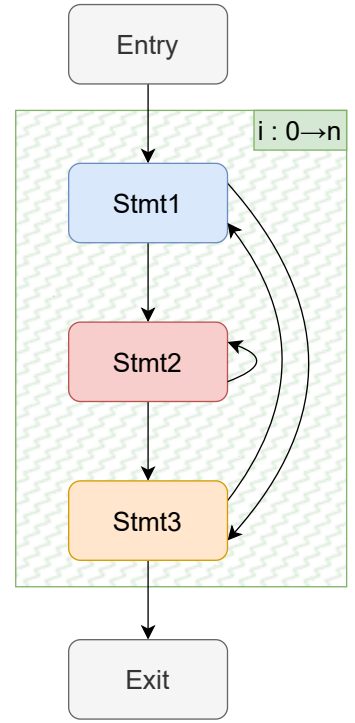
Fewer SCoP parameters → **lower compile time** (scheduling is exponential in the parameters).

Triangular loops written by hand

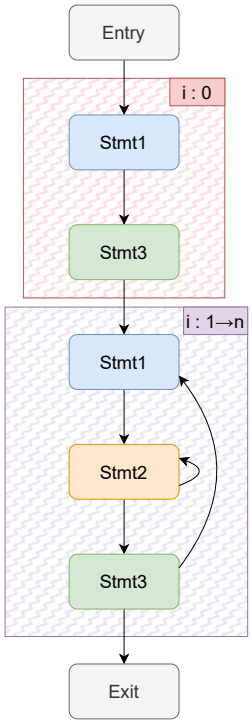
Policies are rectangular, but developers nest C++ loops in the body (trisolv):

```C++
Kokkos::parallel_for<usePolyOpt>(policy,
  KOKKOS_LAMBDA(size_t i) {
    x(i) = b(i);                        // Stmt1
    for (size_t j = 0; j < i; j++)
      x(i) -= L(i, j) * x(j);           // Stmt2
    x(i) = x(i) / L(i, i);              // Stmt3
  });
```

- `i = 0`: empty inner loop, Stmt1 → Stmt3 that breaks SCoP detection.
- SCEV: backedge-taken count is `{-1,+,1}<i>`, not invariant.
- **Peeling** iteration `i = 0` removes the guard → valid SCoP.



Before: irregular branch



After peeling

Scheduling for CPU and GPU

Pluto on CPU, PPCG on GPU

Pluto scheduler integration

- ISL (Polly default) minimizes dependence distances incrementally.
- Pluto targets **tiling and wavefront parallelism** (diamond tiling on stencils).
- Pluto is source-to-source: **OpenScop export/ import bridge** around Polly's SCoP.
- Pluto's tile dimensions re-encoded as **schedule-tree band nodes** (domain kept invariant).

GPU code generation (PPCG)

- **Polly-ACC revived** on the modern Polly.
- Polly's SCoP feeds PPCG: GPU schedule, mapping to **blocks and threads**.
- Polly emits device IR + host code (launches, transfers).
- PPCG exploits **shared memory** and **privatization**.

Evaluation

Experimental setup

Benchmarks : PolyBench rewritten in Kokkos, large sizes.

- **Baseline** : idiomatic Kokkos, original algorithm, outer loops parallelized. Not expert-tuned: we measure what **automatic** scheduling adds.
- 27 of 30 kernels (3 have data-dependent control flow).

Machines

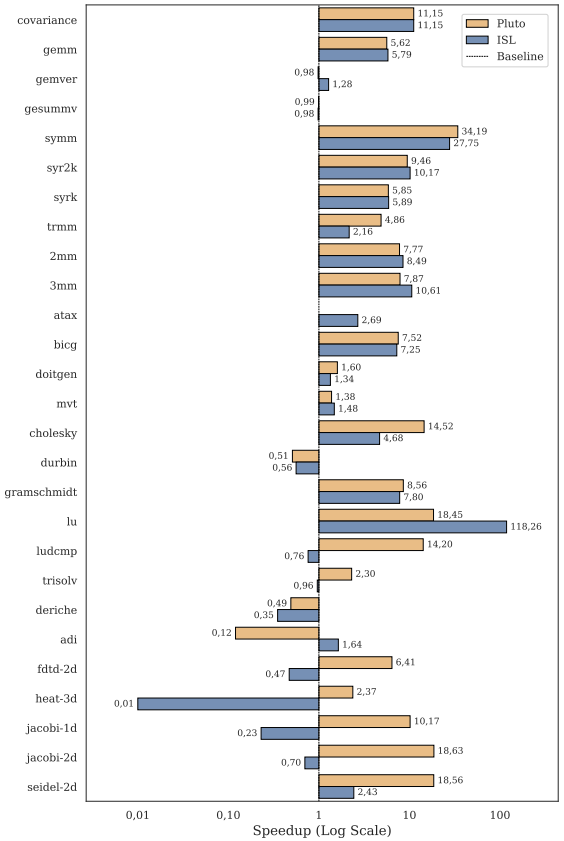
- Dual AMD EPYC 7502 (2 $\times$ 32 cores, 512 KB L2/core).
- Intel Core Ultra 7 265 (20 cores, 3 MB L2/core) + RTX 5090.

Toolchain : Clang/LLVM 23 dev + modified Polly and Kokkos.

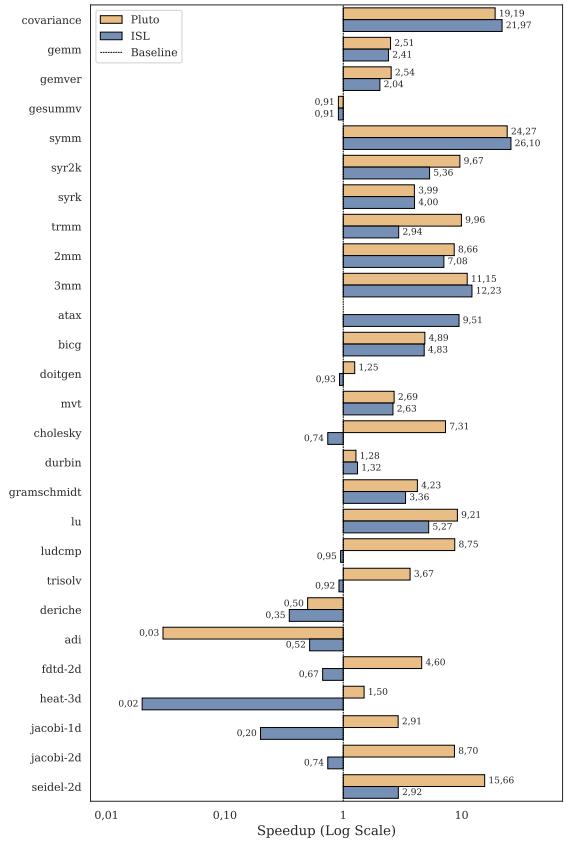
Coverage

- CPU: 53 / 54 variants correct (`atax` + Pluto: bug in Pluto).
- GPU: 24 / 27 (PPCG does not terminate scheduling on 3 stencils).

CPU results: AMD EPYC 7502 and Intel Core Ultra 7 265



Dual AMD EPYC 7502 (64 threads)



Intel Core Ultra 7 265 (20 threads)

Speedup over the Kokkos OpenMP baseline, ISL and Pluto schedulers, tile size 32.

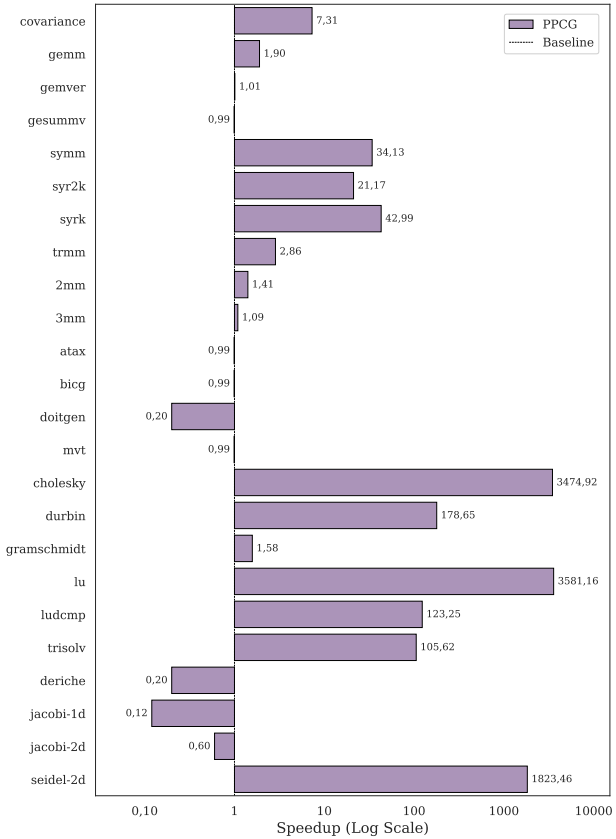
CPU results: summary

	AMD EPYC		Intel Ultra 7	
Category	Pluto	ISL	Pluto	ISL
Datamining (1)	11.15×	11.15×	19.19×	21.97×
BLAS (7)	4.69×	4.27×	4.74×	3.57×
Kernels (6)	3.99×	3.89×	4.37×	4.62×
Solvers (6)	5.80×	3.48×	4.77×	1.57×
Medley (1)	0.49×	0.35×	0.50×	0.35×
Stencils (6)	4.31×	0.38×	2.08×	0.38×
All (27)	**4.44×**	**2.21×**	**3.74×**	**1.88×**

Geometric means of speedups per category.

- **24 of 27 kernels** accelerated on each machine by at least one scheduler.
  - AMD: **5.32×**
  - Intel: **4.36×**
- Two sources of gain:
  - **locality** (baseline parallel but poor reuse)
  - **parallelism** (baseline partly sequential)
- **Pluto wins on stencils and solvers**: Pluto exposes wavefront parallelism and ISL parallelizes the innermost loop

GPU results: NVIDIA RTX 5090 with PPCG



- **16 of 24 kernels** accelerated → geometric mean 7.25×.
- **Solvers**: skewing exposes wavefront parallelism where the baseline keeps a sequential outer loop (`lu`, `cholesky`, `seidel-2d`)
- **Linear algebra**: shared memory and privatization (`syrk` 43×, `symm` 34×)
- **Slowdowns** on light kernels (`doitgen`, `deriche`, `jacobi`): same parallelism as Kokkos, but with polyhedral boundary-check overhead and without Kokkos' launch-parameter heuristics.

Speedup over the Kokkos OpenMP baseline, ISL and Pluto schedulers, tile size 32.

Conclusion

Take-aways

What we showed

- The obstacle is **visibility**; a library–compiler co-design removes it.
- One template option, no application rewrite.
- Delinearization, inter-kernel fusion, triangular loops, Pluto and PPCG now work on Kokkos code.
- Up to **118**× on CPU, **three orders of magnitude** on GPU over idiomatic Kokkos.

Limits and future work

- Runtime checks for user constraints and static names (today trusted as-is).
- Compilation timeouts on complex stencils.
- More Kokkos backends: HIP, SYCL.

Artifact

Docker image (toolchain, benchmarks, scripts): `registry.gitlab.inria.fr/eauer-utils/eauer-docker-images/polly-kokkos/debian12:latest`

Thank you!

Questions?